# Global Academy of Technology, Bengaluru

## Artificial Intelligence and Data Science

### Course: Computer Networks (BAD24404)

### *Module 2: Digital Transmission*

**Digital transmission** is the process of sending **digital data (0s and 1s)** from one device to another through a communication channel such as a **cable, optical fiber, or wireless medium**.

### Example

When you send a **message, email, or file** through the internet, the information is converted into **binary digits (0 and 1)** and transmitted between devices.

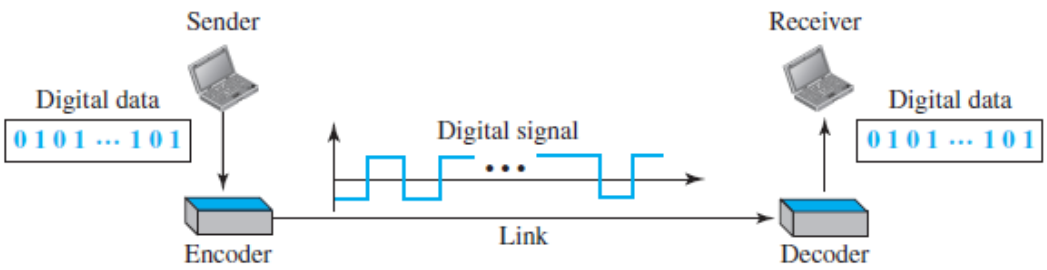
## 2.1 Digital to digital conversion

**Digital-to-Digital Conversion** is a technique used in **data communication** to convert **digital data (0s and 1s)** into **digital signals** so that the data can be transmitted over a communication medium.

***Line Coding***

**Line coding** is the process of converting digital data to digital signals. We assume that data, in the form of text, numbers, graphical images, audio, or video, are stored in computer memory as sequences of bits (see Chapter 1). Line coding converts a sequence of bits to a digital signal. At the sender, digital data are encoded into a digital signal; at the receiver, the digital data are recreated by decoding the digital signal. Figure 4.1 shows the process.

**Figure 4.1** *Line coding and decoding*

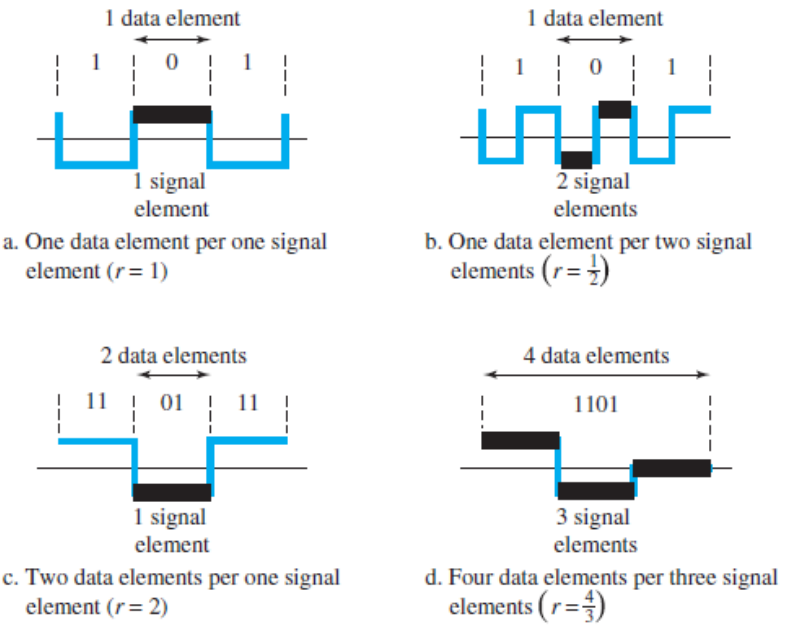


- ***Characteristics***
  - ❖ ***Signal Element Versus Data Element***

In data communications, our goal is to send data elements. A data element is the smallest entity that can represent a piece of information: this is the bit. In digital data communications, a signal element carries data elements. A signal element is the shortest unit (timewise) of a digital signal. In other words, data elements are what we need to send; signal elements are what we can send. Data elements are being carried; signal elements are the carriers. We define a ratio *r* which is the number of data elements carried by each signal element. Figure 4.2 shows several situations with different values of *r*.

In part a of the figure, one data element is carried by one signal element ($r = 1$). In part b of the figure, we need two signal elements (two transitions) to carry each data element ($r = 1/2$). We will see later that the extra signal element is needed to guarantee synchronization. In part c of the figure, a signal element carries two data elements ($r = 2$).

**Figure 4.2** *Signal element versus data element*



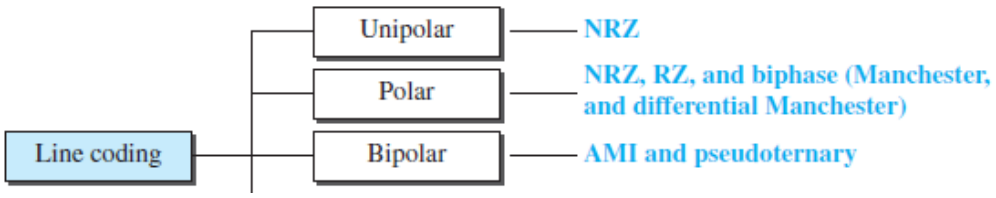

Finally, in part d, a group of 4 bits is being carried by a group of three signal elements ($r$ □□4/3). For every line coding scheme we discuss, we will give the value of *r*. An analogy may help here. Suppose each data element is a person who needs to be carried from one place to another. We can think of a signal element as a vehicle that can carry people. When $r$ □□1, it means each person is driving a vehicle. When $r > 1$, it means more than one person is travelling in a vehicle (a carpool, for example). We can also have the case where one person is driving a car and a trailer ($r$ □□1/2).

### *2.1.2 Line Coding Schemes*

**Line coding schemes** are techniques used in **digital communication** to convert **binary data (0s and 1s)** into **digital signals** that can be transmitted over a communication channel.

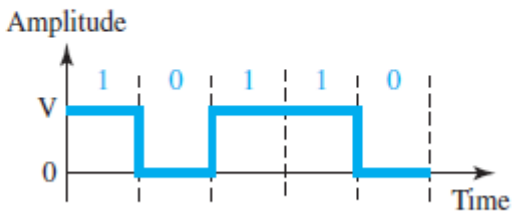
***Unipolar Scheme***
In a **unipolar** scheme, all the signal levels are on one side of the time axis, either above or below.

**Figure 4.4** *Line coding schemes*




***Unipolar Scheme***
In a **unipolar** scheme, all the signal levels are on one side of the time axis, either above or below. ***NRZ (Non-Return-to-Zero)*** Traditionally, a unipolar scheme was designed as a **non-return-to-zero (NRZ)** scheme in which the positive voltage defines bit 1 and the zero voltage defines bit 0. It is called NRZ because the signal does not return to zero at the middle of the bit. Figure 4.5 shows a unipolar NRZ scheme.

**Figure 4.5** *Unipolar NRZ scheme*

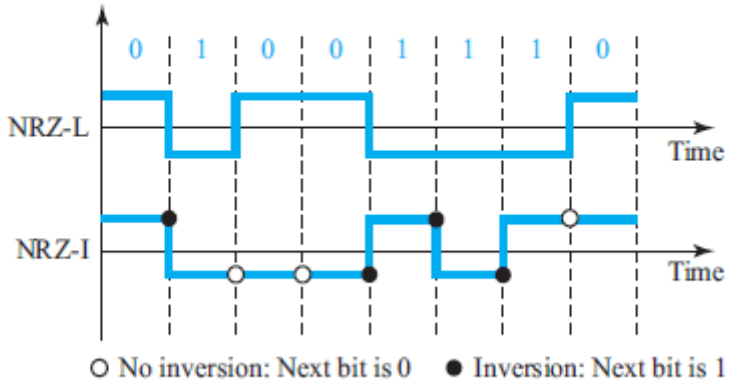



***Polar Schemes***

In **polar** schemes, the voltages are on both sides of the time axis. For example, the voltage level for 0 can be positive and the voltage level for 1 can be negative.

***Non-Return-to-Zero (NRZ)***
In **polar NRZ** encoding, we use two levels of voltage amplitude. We can have two versions of polar NRZ: NRZ-L and NRZ-I, as shown in Figure 4.6. The figure also shows the value of *r*, the average baud rate, and the bandwidth. In the first variation, NRZ-L (**NRZ-Level**), the level of the voltage determines the value of the bit. In the second variation, NRZ-I (**NRZ-Invert**), the change or lack of change in the level of the voltage determines the value of the bit. If there is no change, the bit is 0; if there is a change, the bit is 1.

**Figure 4.6** *Polar NRZ-L and NRZ-I schemes*

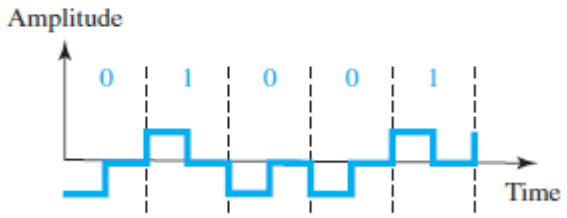



***Return-to-Zero (RZ)***

The main problem with NRZ encoding occurs when the sender and receiver clocks are not synchronized. The receiver does not know when one bit has ended and the next bit is starting. One solution is the **return-to-zero (RZ)** scheme, which uses three values: positive, negative, and zero. In RZ, the signal changes not between bits but during the bit. In Figure 4.7 we see that the signal goes to 0 in the middle of each bit. It remains there until the beginning of the next bit. The main disadvantage of RZ encoding is that it requires two signal changes to encode a bit and therefore occupies greater bandwidth.

The same problem we mentioned, a sudden change of polarity resulting in all 0s interpreted as 1s and all 1s interpreted as 0s, still exists here, but there is no DC component problem. Another problem is the complexity: RZ uses three levels of voltage, which is more complex to create and discern. As a result of all these deficiencies, the scheme is not used today.

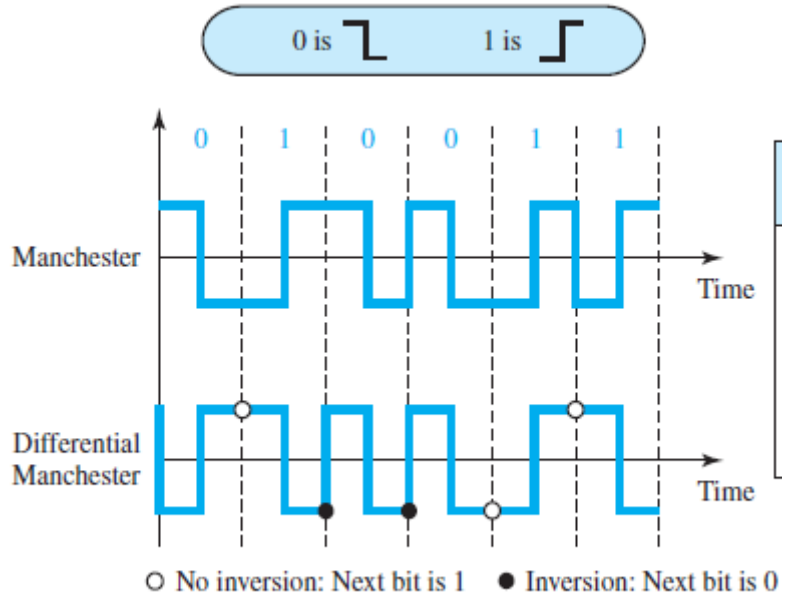
**Figure 4.7** *Polar RZ scheme*



***Biphase: Manchester and Differential Manchester***

The idea of RZ (transition at the middle of the bit) and the idea of NRZ-L are combined into the **Manchester** scheme. In Manchester encoding, the duration of the bit is dividedinto two halves. The voltage remains at one level during the first half and moves to the other level in the second half. The transition at the middle of the bit provides synchronization. **Differential Manchester,** on the other hand, combines the ideas of RZ and NRZ-I. There is always a transition at the middle of the bit, but the bit values are determined at the beginning of the bit. If the next bit is 0, there is a transition; if the next bit is 1, there is none. Figure 4.8 shows both Manchester and differential Manchester encoding.

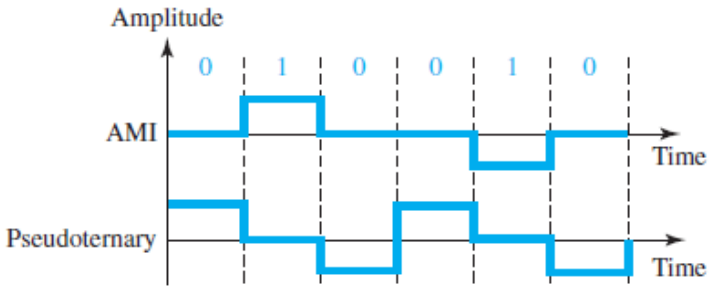
**Figure 4.8** *Polar biphase: Manchester and differential Manchester schemes*




***Bipolar Schemes***

In **bipolar** encoding (sometimes called ***multilevel binary***), there are three voltage levels: positive, negative, and zero. The voltage level for one data element is at zero, while the voltage level for the other element alternates between positive and negative.

**Figure 4.9** *Bipolar schemes: AMI and pseudoternary*



***AMI and Pseudoternary***

Figure 4.9 shows two variations of bipolar encoding: AMI and pseudoternary. A common bipolar encoding scheme is called bipolar **alternate mark inversion (AMI).** In the term *alternate mark inversion*, the word *mark* comes from telegraphy and means 1. So AMI means alternate 1 inversion. A neutral zero voltage represents binary 0. Binary 1s are represented by alternating positive and negative voltages. A variation of AMI encoding is called **pseudoternary** in which the 1 bit is encoded as a zero voltage and the 0 bit is encoded as alternating positive and negative voltages.

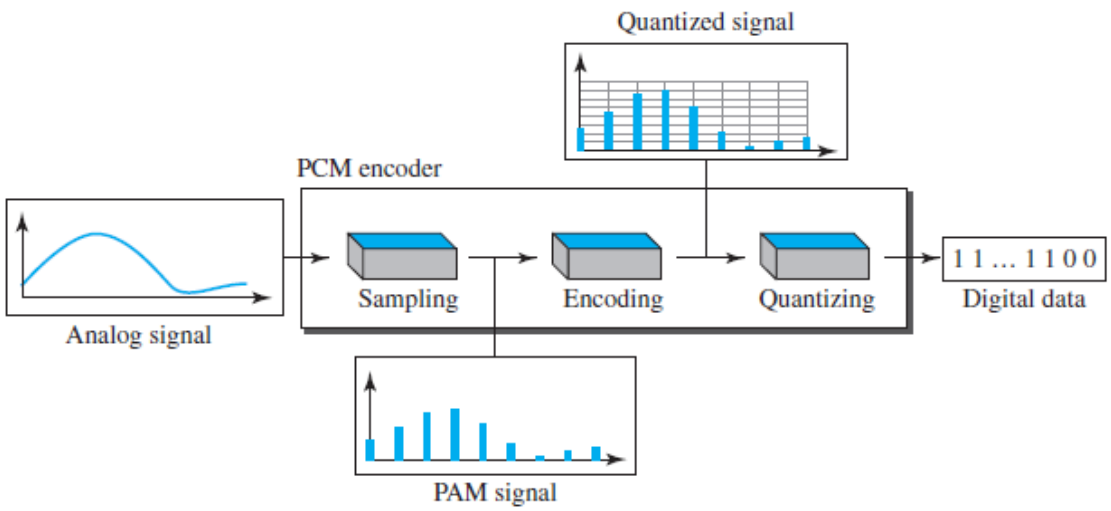
## 2.2 Analog to digital conversion

Sometimes, we have an analog signal such as one created by a microphone or camera.The tendency today is to change an analog signal to digital data , one of the techniques called pulse code modulation.

**Pulse Code Modulation (PCM)**

The most common technique to change an analog signal to digital data (**digitization**) is called **pulse code modulation (PCM).** A PCM encoder has three processes, as shown in Figure 4.21.

**Figure 4.21** *Components of PCM encoder*




**1.** The analog signal is sampled.
**2.** The sampled signal is quantized.
**3.** The quantized values are encoded as streams of bits.

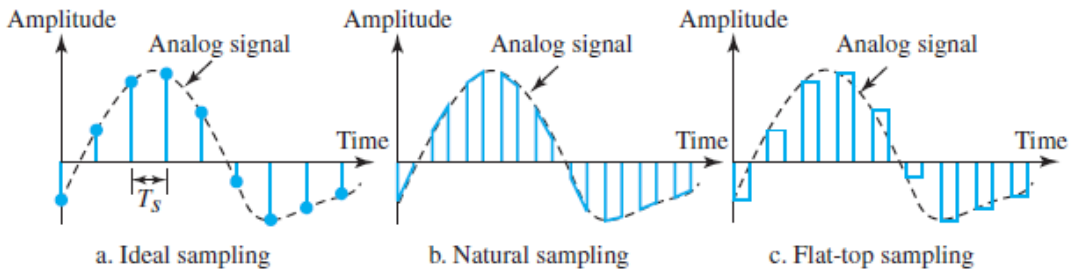
***Sampling***

The first step in PCM is **sampling.** The analog signal is sampled every *Ts* s, where *Ts* is the sample interval or period. The inverse of the sampling interval is called the ***sampling rate*** or ***sampling frequency*** and denoted by *fs*, where *fs* □□1/*Ts*. There are three ampling methods—ideal, natural, and flat-top—as shown in Figure 4.22.

In ideal sampling, pulses from the analog signal are sampled. This is an ideal sampling

method and cannot be easily implemented. In natural sampling, a high-speed switch is turned on for only the small period of time when the sampling occurs. The result is a sequence of samples that retains the shape of the analog signal. The most common sampling method, called ***sample and hold,*** however, creates flat-top samples by using a circuit. The sampling process is sometimes referred to as **pulse amplitude modulation**
**(PAM).** We need to remember, however, that the result is still an analog signal with nonintegral values.

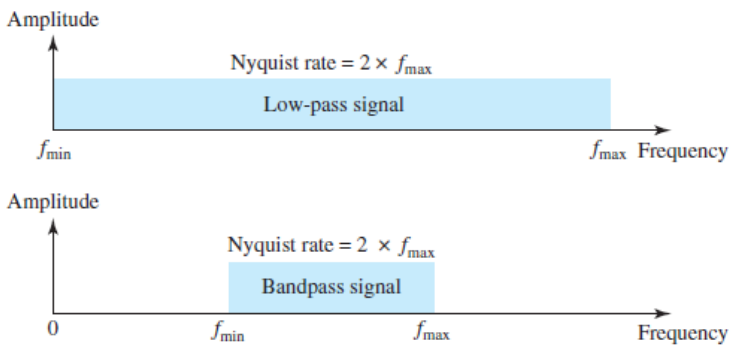
**Figure 4.22** *Three different sampling methods for PCM*




***Sampling Rate***
One important consideration is the sampling rate or frequency. What are the restrictions on *Ts*? This question was elegantly answered by Nyquist. According to the **Nyquist theorem,** to reproduce the original analog signal, one necessary condition is that the *sampling rate* be at least twice the highest frequency in the original signal.

**According to the Nyquist theorem, the sampling rate must be at least 2 times the highest frequency contained in the signal.**

**Figure 4.23** *Nyquist sampling rate for low-pass and bandpass signals*




We need to elaborate on the theorem at this point. First, we can sample a signal only if the signal is band-limited. In other words, a signal with an infinite bandwidth cannot be sampled. Second, the sampling rate must be at least 2 times the highest frequency, not the bandwidth. If the analog signal is low-pass, the bandwidth and the highest frequency are the same value. If

the analog signal is bandpass, the bandwidth value is lower than the value of the maximum frequency. Figure 4.23 shows the value of the sampling rate for two types of signals.

***Quantization***

The result of sampling is a series of pulses with amplitude values between the maximum and minimum amplitudes of the signal. The set of amplitudes can be infinite with nonintegral values between the two limits. These values cannot be used in the encoding process. The following are the steps in quantization:

1. We assume that the original analog signal has instantaneous amplitudes between $V_{min}$ and $V_{max}$.
2. We divide the range into $L$ zones, each of height $\Delta$ (delta).

$$\Delta = \frac{V_{max} - V_{min}}{L}$$

3. We assign quantized values of 0 to $L - 1$ to the midpoint of each zone.
4. We approximate the value of the sample amplitude to the quantized values.

As a simple example, assume that we have a sampled signal and the sample amplitudes are between −20 and +20 V. We decide to have eight levels ($L$ =8). This means that delta=5 V. Figure 4.26 shows this example.

We have shown only nine samples using ideal sampling (for simplicity). The value at the top of each sample in the graph shows the actual amplitude. In the chart, the first row is the normalized value for each sample (actual amplitude/□). The quantization process selects the quantization value from the middle of each zone. This means that the normalized quantized values (second row) are different from the normalized amplitudes. The difference is called the *normalized error* (third row). The fourth row is the quantization code for each sample based on the quantization levels at the left of the graph. The encoded words (fifth row) are the final products of the conversion.

***Quantization Levels***

In the previous example, we showed eight quantization levels. The choice of $L$, the number of levels, depends on the range of the amplitudes of the analog signal and how accurately we need to recover the signal. If the amplitude of a signal fluctuates between two values only, we need only two levels; if the signal, like voice, has many amplitude values, we need more quantization levels. In audio digitizing, $L$ is normally chosen to be 256; in video it is normally thousands. Choosing lower values of $L$ increases the quantization error if there is a lot of fluctuation in the signal.

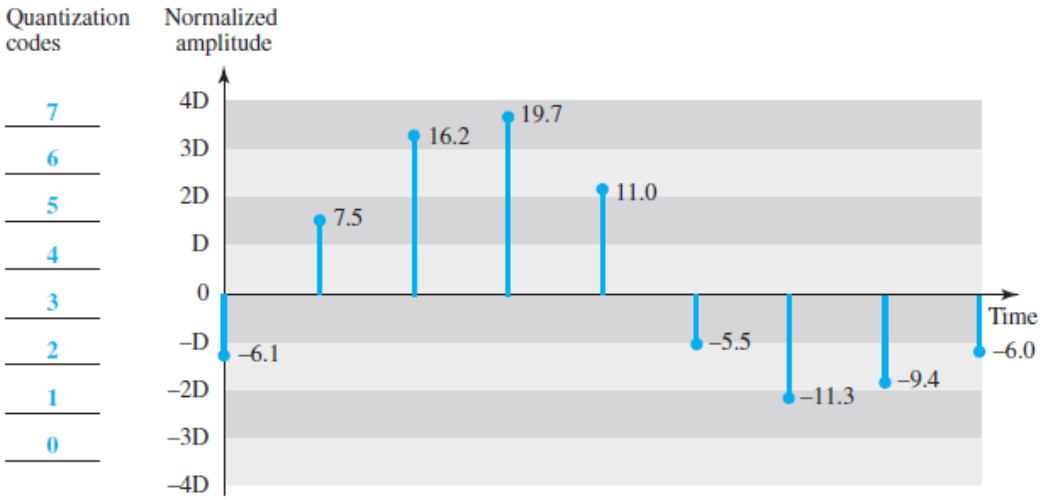
***Quantization Error***

One important issue is the error created in the quantization process. (Later, we will see how this affects high-speed modems.) Quantization is an approximation process. The input values to the quantizer are the real values; the output values are the approximated values. The output values are chosen to be the middle value in the zone. If the input value is also at the middle of

the zone, there is no quantization error; otherwise, there is an error. In the previous example, the normalized amplitude of the third sample is 3.24, but the normalized quantized value is 3.50. This means that there is an error of □0.26. The value of the error for any sample is less than □/2. In other words, we have −□/2 ≤ error ≤□□/2.

The quantization error changes the signal-to-noise ratio of the signal, which in turn reduces the upper limit capacity according to Shannon.

**Figure 4.26** *Quantization and encoding of a sampled signal*




| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| Normalized PAM values | –1.22 | 1.50 | 3.24 | 3.94 | 2.20 | –1.10 | –2.26 | –1.88 | –1.20 |
| Normalized quantized values | –1.50 | 1.50 | 3.50 | 3.50 | 2.50 | –1.50 | –2.50 | –1.50 | –1.50 |
| Normalized error | –0.38 | 0 | +0.26 | –0.44 | +0.30 | –0.40 | –0.24 | +0.38 | –0.30 |
| Quantization code | 2 | 5 | 7 | 7 | 6 | 2 | 1 | 2 | 2 |
| Encoded words | 010 | 101 | 111 | 111 | 110 | 010 | 001 | 010 | 010 |

### *Encoding*

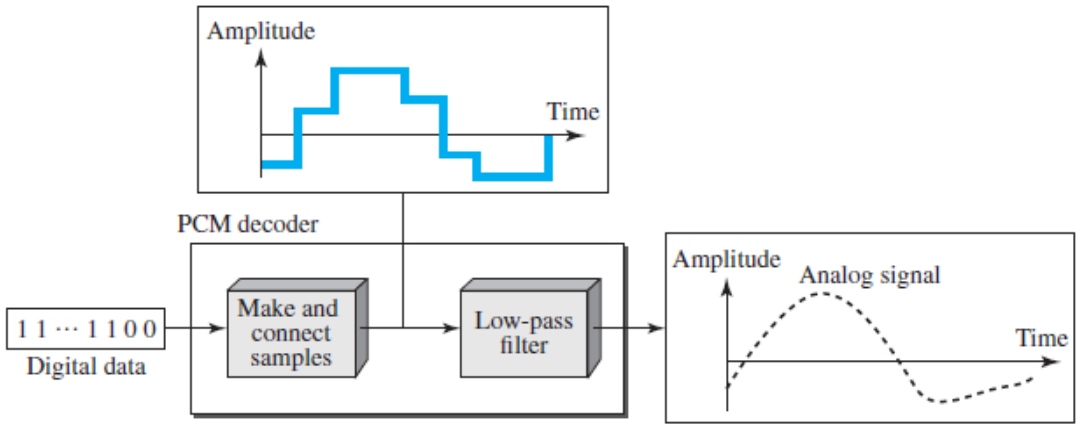
The last step in PCM is encoding. After each sample is quantized and the number of bits per sample is decided, each sample can be changed to an *nb*-bit code word. In Figure 4.26 the encoded words are shown in the last row. A quantization code of 2 is encoded as 010; 5 is encoded as 101; and so on. Note that the number of bits for each sample is determined from the number of quantization levels. If the number of quantization levels is $L$, the number of bits is $nb$ □□log2 $L$. In our example $L$ is 8 and $nb$ is therefore 3.

### *Original Signal Recovery*

The recovery of the original signal requires the PCM decoder. The decoder first uses circuitry to convert the code words into a pulse that holds the amplitude until the next pulse. After the staircase signal is completed, it is passed through a low-pass filter to smooth the staircase signal into an analog signal. The filter has the same cutoff frequency as the original signal at the sender. If the signal has been sampled at (or greater than) the Nyquist sampling rate and if there

are enough quantization levels, the original signal will be recreated. Note that the maximum and minimum values of the original signal can be achieved by using amplification. Figure 4.27 shows the simplified process.

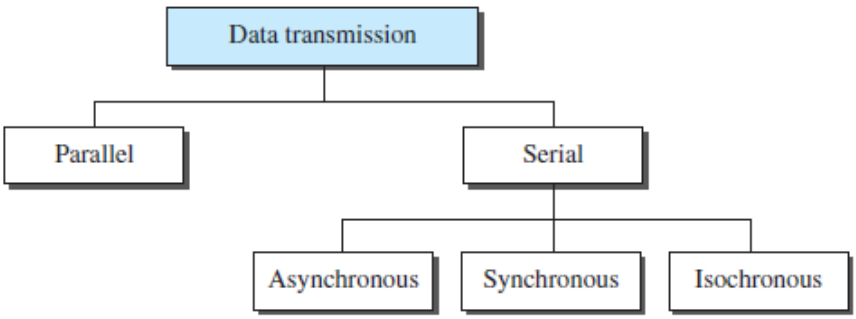
**Figure 4.27** *Components of a PCM decoder*




## 2.3 TRANSMISSION MODES

when we are considering the transmission of data from one device to another is the wiring, and of primary concern when we are considering the wiring is the data stream. Do we send 1 bit at a time; or do we group bits into larger groups and, if so, how? The transmission of binary data across a link can be accomplished in either parallel or serial mode. In parallel mode, multiple bits are sent with each clock tick. In serial mode, 1 bit is sent with each clock tick. While there is only one way to send parallel data, there are three subclasses of serial transmission: asynchronous, synchronous, and isochronous (see Figure 4.31).

**Figure 4.31** *Data transmission and modes*

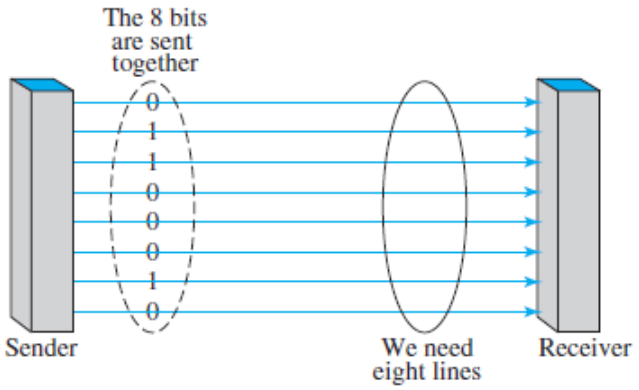



### 2.3.1 Parallel Transmission

Binary data, consisting of 1s and 0s, may be organized into groups of *n* bits each. Computers produce and consume data in groups of bits much as we conceive of and use spoken language in the form of words rather than letters. By grouping, we can send data *n* bits at a time instead of 1. This is called ***parallel transmission.***

The mechanism for parallel transmission is a conceptually simple one: Use *n* wires to send *n* bits at one time. That way each bit has its own wire, and all *n* bits of one group can be transmitted with each clock tick from one device to another. Figure 4.32 shows how parallel transmission works for *n* =8. Typically, the eight wires are bundled in a cable with a connector at each end.

**Figure 4.32** *Parallel transmission*

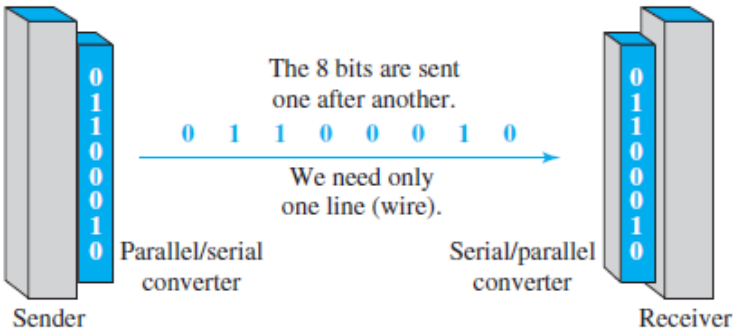



The advantage of parallel transmission is speed. All else being equal, parallel transmission can increase the transfer speed by a factor of *n* over serial transmission. But there is a significant disadvantage: cost. Parallel transmission requires *n* communication lines (wires in the example) just to transmit the data stream. Because this is expensive, parallel transmission is usually limited to short distances.

### 2.3.2 Serial Transmission

In **serial transmission** one bit follows another, so we need only one communication channel rather than *n* to transmit data between two communicating devices (see Figure 4.33).

**Figure 4.33** *Serial transmission*




The advantage of serial over parallel transmission is that with only one communication channel, serial transmission reduces the cost of transmission over parallel by roughly a factor of *n*. Since communication within devices is parallel, conversion devices are required at the interface between the sender and the line (parallel-to-serial) and between the line and the receiver (serial-to-parallel).

Serial transmission occurs in one of three ways: asynchronous, synchronous, and isochronous.

***Asynchronous Transmission***

**Asynchronous transmission** is so named because the timing of a signal is unimportant. Instead, information is received and translated by agreed upon patterns. As long as those patterns are followed, the receiving device can retrieve the information without regard to the rhythm in which it is sent. Patterns are based on grouping the bit stream into bytes. Each group, usually 8 bits, is sent along the link as a unit. The sending system handles each group independently, relaying it to the link whenever ready, without regard to a timer.

Without synchronization, the receiver cannot use timing to predict when the next group will arrive. To alert the receiver to the arrival of a new group, therefore, an extra bit is added to the beginning of each byte. This bit, usually a 0, is called the **start bit.** To let the receiver know that the byte is finished, 1 or more additional bits are appended to the end of the byte. These bits, usually 1s, are called **stop bits.** By this method, each byte is increased in size to at least 10 bits, of which 8 bits is information and 2 bits or more are signals to the receiver. In addition, the transmission of each byte may then be followed by a gap of varying duration. This gap can be represented either by an idle channel or by a stream of additional stop bits.

**In asynchronous transmission, we send 1 start bit (0) at the beginning and 1 or more stop bits (1s) at the end of each byte. There may be a gap between bytes.**

The start and stop bits and the gap alert the receiver to the beginning and end of each byte and allow it to synchronize with the data stream. This mechanism is called asynchronous because, at the byte level, the sender and receiver do not have to be synchronized. But within each byte, the receiver must still be synchronized with the incoming bit stream. That is, some synchronization is required, but only for the duration of a single byte. The receiving device resynchronizes at the onset of each new byte.

When the receiver detects a start bit, it sets a timer and begins counting bits as they come in. After n bits, the receiver looks for a stop bit. As soon as it detects the stop bit, it waits until it detects the next start bit.

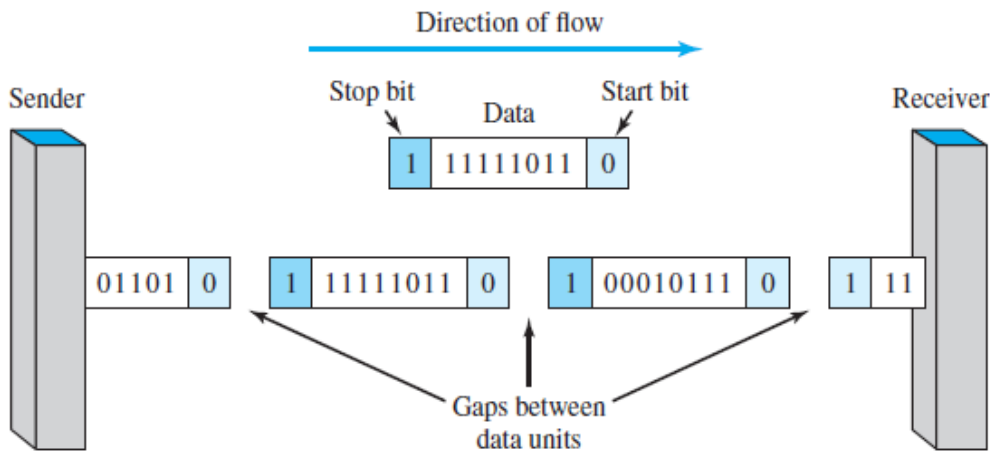
**Asynchronous here means "asynchronous at the byte level," but the bits are still synchronized; their durations are the same.**

Figure 4.34 is a schematic illustration of asynchronous transmission. In this example, the start bits are 0s, the stop bits are 1s, and the gap is represented by an idle line rather than by additional stop bits. The addition of stop and start bits and the insertion of gaps into the bit stream make asynchronous transmission slower than forms of transmission that can operate.

**Synchronous Transmission**

In synchronous transmission, the bit stream is combined into longer "frames," which may contain multiple bytes. Each byte, however, is introduced onto the transmission link without a gap between it and the next one. It is left to the receiver to separate the bit stream into bytes for decoding purposes. In other words, data are transmitted as an unbroken string of 1s and 0s, and the receiver separates that string into the bytes, or characters, it needs to reconstruct the information.

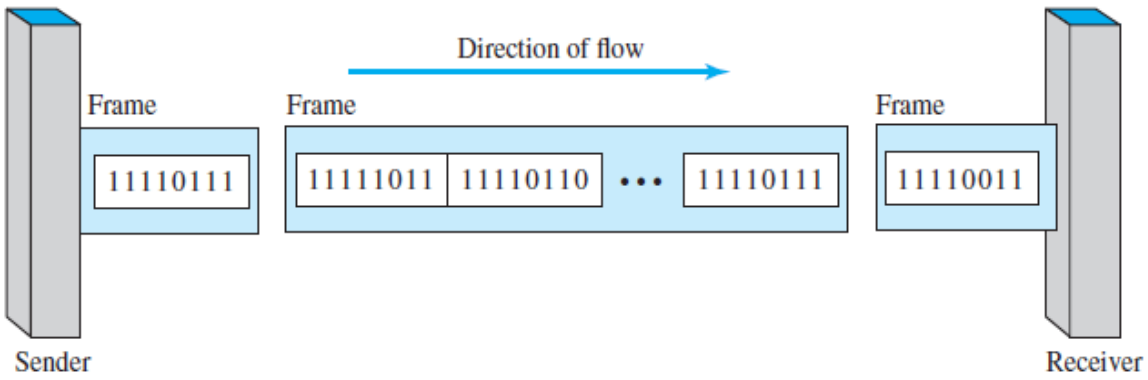
**Figure 4.34** *Asynchronous transmission*




**In synchronous transmission, we send bits one after another without start or stop bits or gaps. It is the responsibility of the receiver to group the bits.**

Figure 4.35 gives a schematic illustration of synchronous transmission. We have drawn in the divisions between bytes. In reality, those divisions do not exist; the sender puts its data onto the line as one long string. If the sender wishes to send data in separate bursts, the gaps between bursts must be filled with a special sequence of 0s and 1s that means *idle*. The receiver counts the bits as they arrive and groups them in 8-bit units. Without gaps and start and stop bits, there is no built-in mechanism to help the receiving device adjust its bit synchronization midstream. Timing becomes very important, therefore, because the accuracy of the received information is completely dependent on the ability of the receiving device to keep an accurate count of the bits as they come in.

**Figure 4.35** *Synchronous transmission*




The advantage of synchronous transmission is speed. With no extra bits or gaps to introduce at the sending end and remove at the receiving end, and, by extension, with fewer bits to move

across the link, synchronous transmission is faster than asynchronous transmission. For this reason, it is more useful for high-speed applications such as the transmission of data from one computer to another. Byte synchronization is accomplished in the data-link layer.

We need to emphasize one point here. Although there is no gap between characters in synchronous serial transmission, there may be uneven gaps between frames.

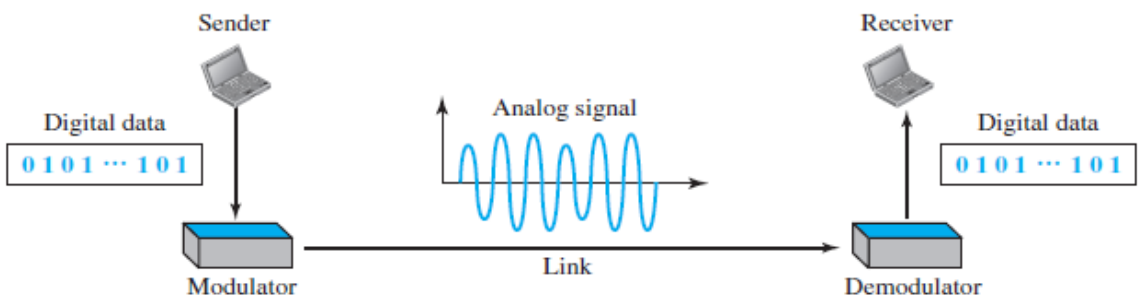
### Isochronous

In real-time audio and video, in which uneven delays between frames are not acceptable, synchronous transmission fails. For example, TV images are broadcast at the rate of 30 images per second; they must be viewed at the same rate. If each image is sent by using one or more frames, there should be no delays between frames. For this type of application, synchronization between characters is not enough; the entire stream of bits must be synchronized. The isochronous transmission guarantees that the data arrive at a fixed rate.

## 2.4 DIGITAL-TO-ANALOG CONVERSION

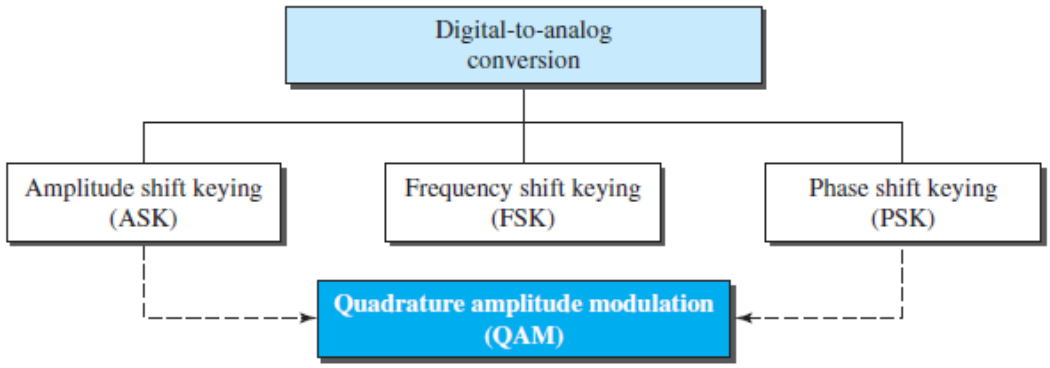
**Digital-to-analog conversion** is the process of changing one of the characteristics of an analog signal based on the information in digital data. Figure 5.1 shows the relationship between the digital information, the digital-to-analog modulating process, and the resultant analog signal.

**Figure 5.1** *Digital-to-analog conversion*




sine wave is defined by three characteristics: amplitude, frequency, and phase. When we vary any one of these characteristics, we create a different version of that wave. So, by changing one characteristic of a simple electric signal, we can use it to represent digital data. Any of the three characteristics can be altered in this way, giving us at least three mechanisms for modulating digital data into an analog signal: **amplitude shift keying (ASK), frequency shift keying (FSK),** and **phase shift keying (PSK).** In addition, there is a fourth (and better) mechanism that combines changing both the amplitude and phase, called **quadrature amplitude modulation (QAM).** QAM is the most efficient of these options and is the mechanism commonly used today (see Figure 5.2).

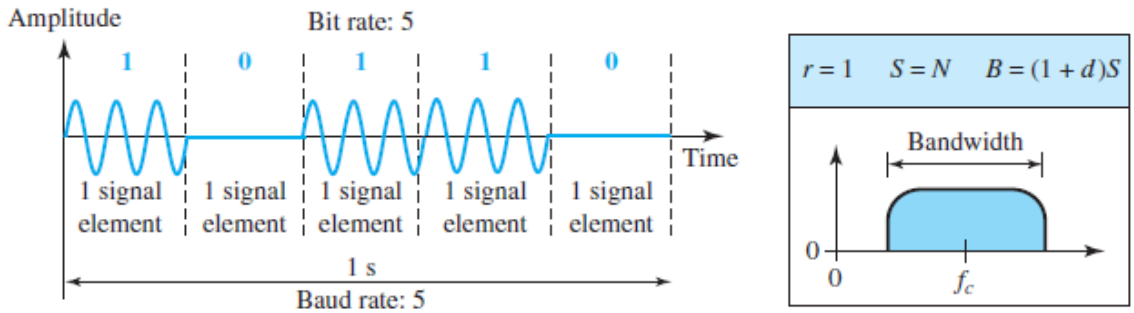
**Figure 5.2** *Types of digital-to-analog conversion*




### 2.4.1 **Amplitude Shift Keying**

In amplitude shift keying, the amplitude of the carrier signal is varied to create signal elements. Both frequency and phase remain constant while the amplitude changes. ***Binary ASK (BASK)*** Although we can have several levels (kinds) of signal elements, each with a different amplitude, ASK is normally implemented using only two levels. This is referred to as *binary amplitude shift keying* or *on-off keying* (OOK). The peak amplitude of one signal level is 0; the other is the same as the amplitude of the carrier frequency. Figure 5.3 gives a conceptual view of binary ASK.

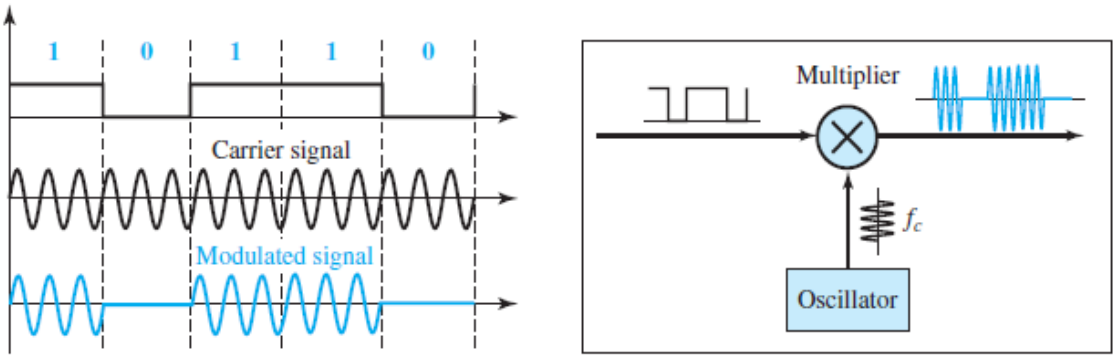
**Figure 5.3** *Binary amplitude shift keying*




***Implementation***

However, the simple ideas behind the implementation may help us to better understand the concept itself. Figure 5.4 shows how we can simply implement binary ASK.

**Figure 5.4** *Implementation of binary ASK*



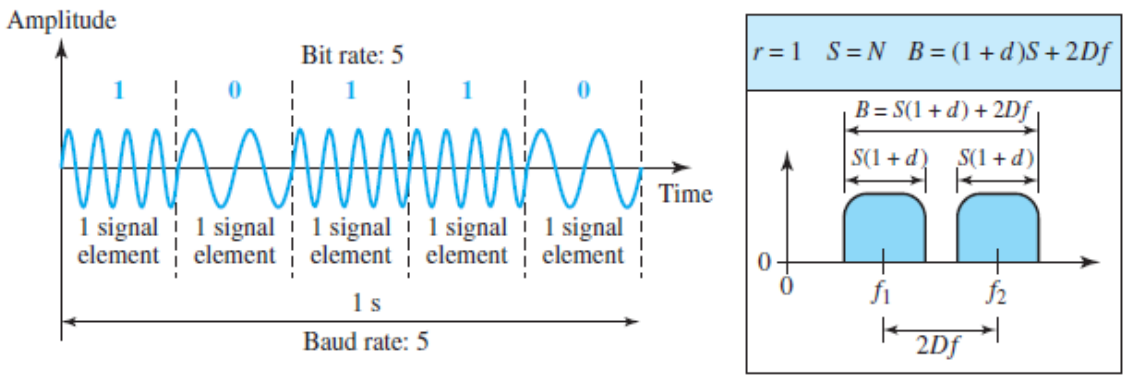

2.4.2 **Frequency Shift Keying**

In frequency shift keying, the frequency of the carrier signal is varied to represent data. The frequency of the modulated signal is constant for the duration of one signal element, but changes for the next signal element if the data element changes. Both peak amplitude and phase remain constant for all signal elements.

***Binary FSK (BFSK)***

One way to think about binary FSK (or BFSK) is to consider two carrier frequencies. In Figure 5.6, we have selected two carrier frequencies, $f1$ and $f2$. We use the first carrier if the data element is 0; we use the second if the data element is 1. However, note that this is an unrealistic example used only for demonstration purposes. Normally the carrier frequencies are very high, and the difference between them is very small.

**Figure 5.6** *Binary frequency shift keying*

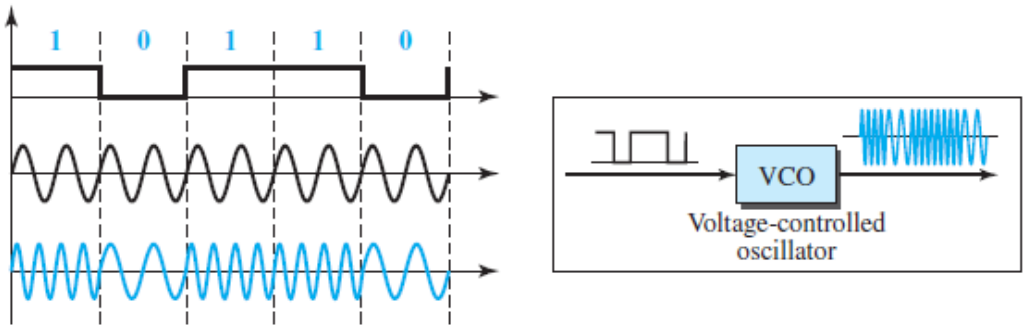



***Implementation***

There are two implementations of BFSK: noncoherent and coherent. In noncoherent BFSK, there may be discontinuity in the phase when one signal element ends and the next begins. In coherent BFSK, the phase continues through the boundary of two signal elements. Noncoherent BFSK can be implemented by treating BFSK as two ASK modulations and using two carrier frequencies. Coherent BFSK can be implemented by using one *voltage-controlled oscillator* (VCO) that changes its frequency according to the input voltage. Figure 5.7 shows the simplified idea behind the second implementation. The input to the oscillator is the unipolar

NRZ signal. When the amplitude of NRZ is zero, the oscillator keeps its regular frequency; when the amplitude is positive, the frequency is increased.

**Figure 5.7** *Implementation of BFSK*




### 2.4.3 Phase Shift Keying

In phase shift keying, the phase of the carrier is varied to represent two or more different signal elements. Both peak amplitude and frequency remain constant as the phase changes. Today, PSK is more common than ASK or FSK. However, we will see shortly that QAM, which combines ASK and PSK, is the dominant method of digital-to-analog modulation.

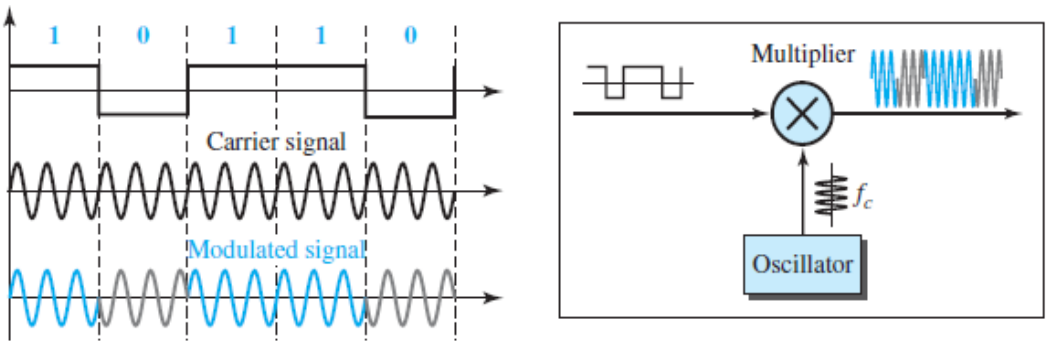
***Binary PSK (BPSK)***

The simplest PSK is binary PSK, in which we have only two signal elements, one with a phase of 0°, and the other with a phase of 180°. Figure 5.9 gives a conceptual view of PSK. Binary PSK is as simple as binary ASK with one big advantage it is less susceptible to noise. In ASK, the criterion for bit detection is the amplitude of the signal; in PSK, it is the phase. Noise can change the amplitude easier than it can change the phase. In other words, PSK is less susceptible to noise than ASK. PSK is superior to FSK because we do not need two carrier signals. However, PSK needs more sophisticated hardware to be able to distinguish between phases.

***Implementation***

The implementation of BPSK is as simple as that for ASK. The reason is that the signal element with phase 180° can be seen as the complement of the signal element with phase 0°. This gives us a clue on how to implement BPSK. We use the same idea we used for ASK but with a polar NRZ signal instead of a unipolar NRZ signal, as shown in Figure 5.10. The polar NRZ signal is multiplied by the carrier frequency; the 1 bit (positive voltage) is represented by a phase starting at 0°; the 0 bit (negative voltage) is represented by a phase starting at 180°.

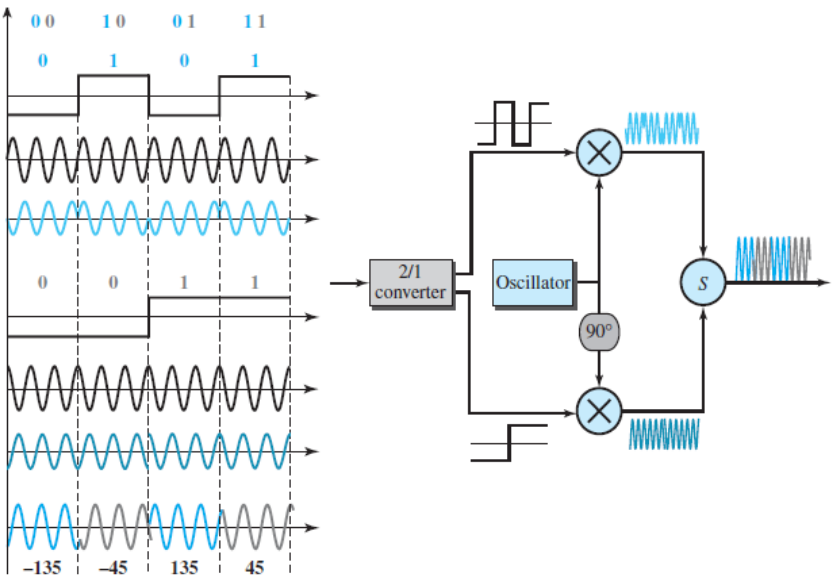
**Figure 5.10** *Implementation of BASK*




### *Quadrature PSK (QPSK)*

The simplicity of BPSK enticed designers to use 2 bits at a time in each signal element, thereby decreasing the baud rate and eventually the required bandwidth. The scheme is called *quadrature PSK* or *QPSK* because it uses two separate BPSK modulations; one is in-phase, the other quadrature (out-of-phase). The incoming bits are first passed through a serial-to-parallel conversion that sends one bit to one modulator and the next bit to the other modulator. If the duration of each bit in the incoming signal is $T$, the duration of each bit sent to the corresponding BPSK signal is $2T$. This means that the bit to each BPSK signal has one-half the frequency of the original signal. Figure 5.11 shows the idea.

**Figure 5.11** *QPSK and its implementation*




The two composite signals created by each multiplier are sine waves with the same frequency, but different phases. When they are added, the result is another sine wave, with one of four possible phases: 45°, −45°, 135°, and −135°. There are four kinds of signal elements in the output signal ($L = 4$), so we can send 2 bits per signal element ($r = 2$).